

Aulão de Banco de Dados

Agrupar e agregar — pensar em conjuntos

Tópicos Especiais II · ENADE · 2026/2

Prof. M.Sc. Howard Cruz Roatti

A pergunta de hoje

Você já escreve `SELECT`, `WHERE` e `JOIN`. Aí chega esta pergunta simples:



"Temos uma tabela com **uma venda por linha**. Quanto **cada loja** faturou?"

- A consulta **linha a linha** não responde: ela devolve **as vendas**, não **o total por loja**.
- Falta uma ferramenta para **resumir muitos registros em poucos**.

🎯 Hoje a gente monta essa ferramenta do zero: `GROUP BY`, funções de agregação, `HAVING` e `ORDER BY`.
No fim, você resolve uma questão de ENADE sem esforço.

A virada de chave

Jeito procedural (de quem programa)

Percorrer **uma linha de cada vez** e ir somando numa variável.

```
for venda in vendas: total[loja] += venda
```

Jeito do SQL (declarativo)

Você **não percorre** nada. Descreve **grupos** e o que quer de **cada grupo**.

"Para **cada loja**, a **soma** dos valores."

💡 A ideia-mãe: a tabela é um **monte de linhas** → você as separa em **baldes** (grupos) → e pede **um número por balde**. Pensar em **conjuntos**, não em laços.

Nossa tabela de exemplo — VENDAS

loja	vendedor	valor
Norte	Ana	100
Norte	Bia	50
Sul	Caio	200
Sul	Duda	80
Leste	Edu	30

Uma venda por linha.

Cinco linhas, três lojas.

Vamos responder, passo a passo:

1. quanto faturou **no total**?
2. quanto faturou **cada loja**?
3. e **só** as lojas grandes?
4. da **maior** para a menor?

Passo 1 — resumir a tabela inteira

Uma **função de agregação** engole **muitas linhas** e devolve **uma**:

```
SELECT SUM(valor), COUNT(*), AVG(valor) FROM VENDAS;
```

SUM(valor)	COUNT(*)	AVG(valor)
460	5	92

💡 Sem GROUP BY, a tabela toda é **um único grupo** → uma linha de resposta. SUM soma, COUNT conta, AVG tira a média, MIN / MAX pegam extremos.

Passo 2 — GROUP BY: um resumo por grupo

GROUP BY loja separa as linhas em **baldes** e aplica a função em **cada um**:

Entram (agrupadas por cor)

loja	valor
Norte	100
Norte	50
Sul	200
Sul	80
Leste	30

Saem — GROUP BY loja + SUM

loja	SUM(valor)
Norte	150
Sul	280
Leste	30



Cada **cor** vira **uma linha**. Cinco linhas entraram, três saíram — uma por grupo.

A regra de ouro (ela nasce aqui)

No `SELECT`, **cada coluna** ou está no `GROUP BY`, ou está **dentro de uma função** de agregação.

```
SELECT loja, SUM(valor) FROM VENDAS GROUP BY loja; --  Loja agrupa, valor agrega
```

E se eu jogar `valor` no `GROUP BY` também?

```
SELECT loja, SUM(valor) FROM VENDAS GROUP BY loja, valor; -- 
```

 Agora cada **par (loja, valor)** é um grupo → volta a **uma linha por valor**, e a soma vira o próprio valor. A totalização **some** — e o banco aceita numa boa. Por isso a regra: **agrupa-se pelo que NÃO está agregado**.

Vote no ar 🖐️ — contexto novo, mesma ideia

Nova tabela GOLS(time, jogador, gols):

time	jogador	gols
Leões	Rui	2
Leões	Téo	1
Tigres	Vin	3
Tigres	Zeca	1

Qual dá o **total de gols por TIME**?

```
-- 1
SELECT time, SUM(gols)
FROM GOLS
GROUP BY time;

-- 2
SELECT time, SUM(gols)
FROM GOLS
GROUP BY time, jogador;
```

🖐️ 1 ou 2? Antes de decidir: o que é "um balde" aqui?

Vote no ar 🖐️ — a resposta

Consulta 1 — GROUP BY time ✅

time	SUM(gols)
Leões	3
Tigres	4

Um balde por **time** → total **por time**.

Consulta 2 — GROUP BY time, jogador ❌

time	jogador	SUM
Leões	Rui	2
Leões	Téo	1
Tigres	Vin	3
Tigres	Zeca	1

💡 Pôr jogador no GROUP BY **muda o balde** → total por jogador, não por time. A regra de novo: agrupe-se só pelo que **não está agregado** (aqui, só time).

Passo 3 — WHERE × HAVING

Os dois filtram — mas em **momentos diferentes**:

WHERE → filtra **LINHAS**

antes de agrupar.

```
WHERE valor >= 50
```

(descarta a venda de 30 **antes** da soma)

HAVING → filtra **GRUPOS**

depois de agrupar.

```
HAVING SUM(valor) > 100
```

(descarta a loja cujo **total** ≤ 100)

```
SELECT loja, SUM(valor) FROM VENDAS
GROUP BY loja
HAVING SUM(valor) > 100;      -- só Norte (150) e Sul (280); Leste (30) cai
```

💡 Regra prática: condição sobre **coluna** → WHERE ;
condição sobre **resultado de agregação** (SUM , COUNT ...) → HAVING .

Passo 4 — ORDER BY pela agregação

Dá para ordenar **pelo número que você calculou**:

```
SELECT loja, SUM(valor) AS total FROM VENDAS  
GROUP BY loja  
ORDER BY total DESC;           -- Sul 280, Norte 150, Leste 30
```

loja	total
Sul	280
Norte	150
Leste	30

💡 ORDER BY SUM(valor) DESC OU ORDER BY total DESC (pelo apelido) — os dois funcionam.

A consulta inteira — ordem de escrita × de execução

```
SELECT  loja, SUM(valor) AS total      -- 5º: escolhe o que aparece
FROM    VENDAS                       -- 1º: de onde vêm as linhas
WHERE   valor >= 50                   -- 2º: filtra LINHAS
GROUP BY loja                        -- 3º: forma os GRUPOS
HAVING  SUM(valor) > 100              -- 4º: filtra GRUPOS
ORDER BY total DESC;                 -- 6º: ordena o resultado
```

📌 Você **escreve** nessa ordem, mas o banco **executa** assim: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. É por isso que o HAVING pode usar SUM e o WHERE não.

Agora vocês

Em grupos — 8 minutos

Desafios — em grupos

Tabela VENDAS :

loja	vendedor	valor
Norte	Ana	100
Norte	Bia	50
Sul	Caio	200
Sul	Duda	80
Leste	Edu	30

Escrevam a consulta:

1. **Quantas vendas** cada vendedor fez? (`COUNT`)
2. **Faturamento por loja**, só as que passaram de **100**.
3. A **loja campeã** (*uma linha só*).



Comecem pela pergunta "**o que é um grupo aqui?**" — é o que vai no `GROUP BY` .

Desafios — gabarito

-- 1) vendas por vendedor

```
SELECT vendedor, COUNT(*) FROM VENDAS GROUP BY vendedor;
```

-- 2) faturamento por loja, acima de 100

```
SELECT loja, SUM(valor) FROM VENDAS  
GROUP BY loja HAVING SUM(valor) > 100;
```

-- 3) Loja campeã

```
SELECT loja, SUM(valor) AS total FROM VENDAS  
GROUP BY loja ORDER BY total DESC  
LIMIT 1;                -- (no Oracle: FETCH FIRST 1 ROW ONLY)
```

💡 Reparem: **o que muda** entre eles é só o `GROUP BY` e o filtro/ordenação — a estrutura é a mesma.

Como isso cai no ENADE

Banco de eleição: `Partido(numero, nome)`, `Votacao(partido, votos, estado)`. Queremos o **total de votos por partido**. Assinale a consulta correta.

```
-- A
SELECT p.nome, SUM(v.votos) FROM Partido p, Votacao v
WHERE v.partido = p.numero GROUP BY p.nome;

-- B
SELECT p.nome, SUM(v.votos) FROM Partido p, Votacao v
WHERE v.partido = p.numero GROUP BY p.nome, v.votos;
```

💡 É a **mesma ideia** de hoje: SUM totaliza e o GROUP BY leva só as colunas **não agregadas**. Agrupar por v.votos (B) quebraria a soma → a correta é **A**.

Bloco 2 · Junção

Combinar tabelas — pensar em conjuntos

Junção como conjunto

Combinar linhas de **duas tabelas** pela condição de **casamento** (ON):

CLIENTE

id	nome
1	Ana
2	Bruno
3	Caio

PEDIDO

id	id_cliente	valor
10	1	150
11	1	80
12	2	200

 **Caio não pediu nada.** INNER JOIN = só quem **casa** (3 linhas; Caio fica de fora). CLIENTE LEFT JOIN PEDIDO = **todos os clientes** + o que casar (4 linhas; Caio com pedido NULL). RIGHT é o LEFT ao contrário.

Agora vocês — junção

Com as tabelas `CLIENTE` e `PEDIDO` acima:

1. **Todos os clientes** e seus pedidos — **inclusive quem não pediu**.
2. Só os clientes que **nunca pediram**.
3. **Quanto cada cliente gastou** no total (*inclusive quem gastou 0*).

 "aparecer mesmo sem casar" → `LEFT JOIN` . "não casou" → `... WHERE p.id IS NULL` .

Junção — gabarito

```
-- 1) todos os clientes + pedidos (Caio aparece com NULL)
SELECT c.nome, p.id, p.valor
FROM CLIENTE c LEFT JOIN PEDIDO p ON p.id_cliente = c.id;

-- 2) clientes que nunca pediram → só Caio
SELECT c.nome
FROM CLIENTE c LEFT JOIN PEDIDO p ON p.id_cliente = c.id
WHERE p.id IS NULL;

-- 3) total por cliente (junção + agrupamento!)
SELECT c.nome, COALESCE(SUM(p.valor), 0) AS total
FROM CLIENTE c LEFT JOIN PEDIDO p ON p.id_cliente = c.id
GROUP BY c.nome;
```

💡 O item 3 **junta os dois temas de hoje**: junção + agregação. É assim que cai de verdade.

Bloco 3 · Modelagem

Da cardinalidade à chave estrangeira

Cardinalidade vira chave estrangeira

(mín, máx) : o **primeiro** é o mínimo, o **segundo** o máximo.

- (1,1) = obrigatório e único
- (0,1) = **opcional** e único
- (0,N) / (1,N) = vários
- No **mínimo 0**, a FK **aceita nulo**; no mínimo 1, a FK é **NOT NULL**.

⚠ Pegadinha clássica: ler o mínimo **0** de (0,1) como se fosse obrigatório. "No máximo uma" **não** é "exatamente uma".

Agora vocês — modelagem 🤝

As mesmas tabelas de antes, agora pelo **modelo**:

"Um **PEDIDO** tem **exatamente um** cliente; um **CLIENTE** tem de **0 a N** pedidos."

1. Escreva a cardinalidade (mín, máx) de **cada lado**.
2. Em **qual tabela** entra a **FK**?
3. Essa FK **aceita nulo**?

Modelagem — gabarito

1. **PEDIDO (1,1) · CLIENTE (0,N)** → é um **1:N**.
2. A FK vai no lado **N**: `PEDIDO.id_cliente`.
3. **Não** — mínimo 1 (todo pedido tem cliente) → **NOT NULL**.

💡 É a **mesma** `CLIENTE`/`PEDIDO` do exercício de junção: **modelagem e SQL são o mesmo problema**, visto de dois ângulos.

Fecho — leve isto na prova

💡 **Agregação em uma frase:** forme os grupos com `GROUP BY`, resuma cada um com uma função (`SUM` / `AVG` / `COUNT`), **agrupe pelo que não está agregado**, filtre grupos com `HAVING` e ordene com `ORDER BY`.

Para revisar por conta — decks de BD:

- **Normalização** (muito recorrente!)
- **Transações e ACID**
- **Concorrência / isolamento**
- **Índices** (Views, SQL e Indexação)
- **Álgebra relacional**
- **NoSQL e CAP**

Bons estudos!

howard.cruz@faesa.br

≡ Índice

Tópicos Especiais II

Aula 2 · Banco de Dados

revisão completa